



U.S. General Services
Administration

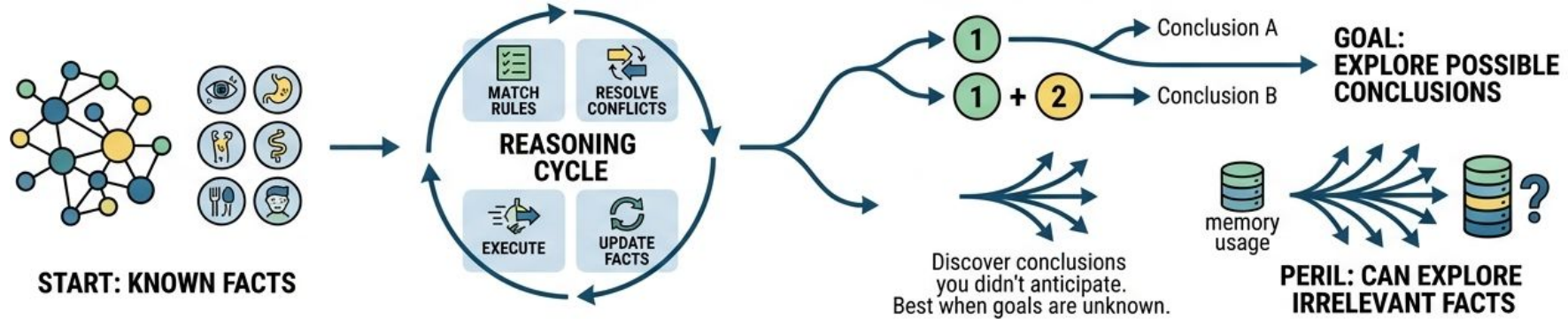
Chapter 5.11: Rule-Based Decision Making Fundamentals

US General Service Administration
AI Community of Practice - March 2026

The Four Components of a Rule-Based System

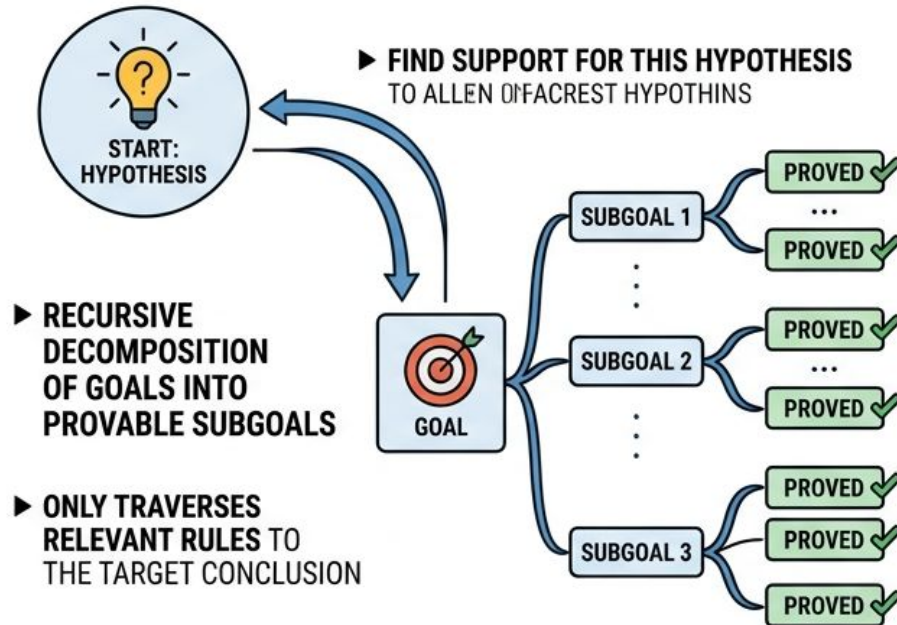
- Knowledge base stores rules and reference facts
 - Inference engine matches rules to facts and executes actions
 - Working memory holds temporary, session-specific state
 - Rule interpreter translates human-readable rules to executable form
 - Separation of concerns keeps the system maintainable
- 

Forward Chaining -- Start With What You Know



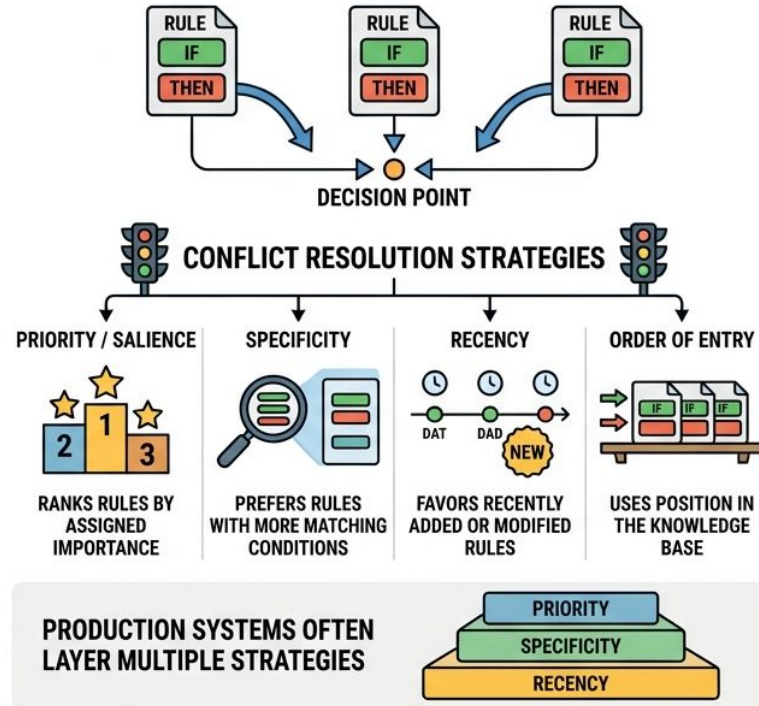
- Begins with known facts, derives conclusions iteratively
- Also called data-driven reasoning
- Each cycle: match rules, resolve conflicts, execute, update facts
- Explores all possible conclusions from available evidence
- Best when goals are unknown or multiple outcomes possible

Backward Chaining



- Begins with a hypothesis, works backward to find support
- Also called goal-driven reasoning
- Recursively decomposes goals into provable subgoals
- Only traverses rules relevant to the target conclusion
- Best when verifying a specific diagnosis or claim

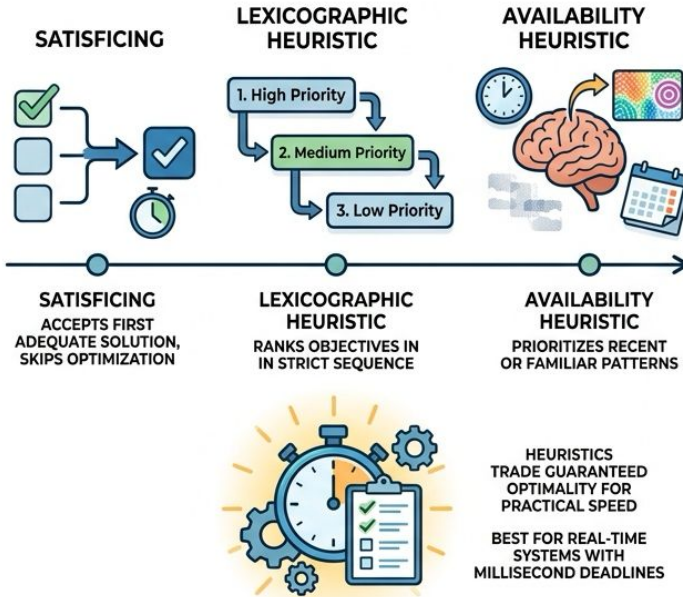
Conflict Resolution -- When Multiple Rules Match



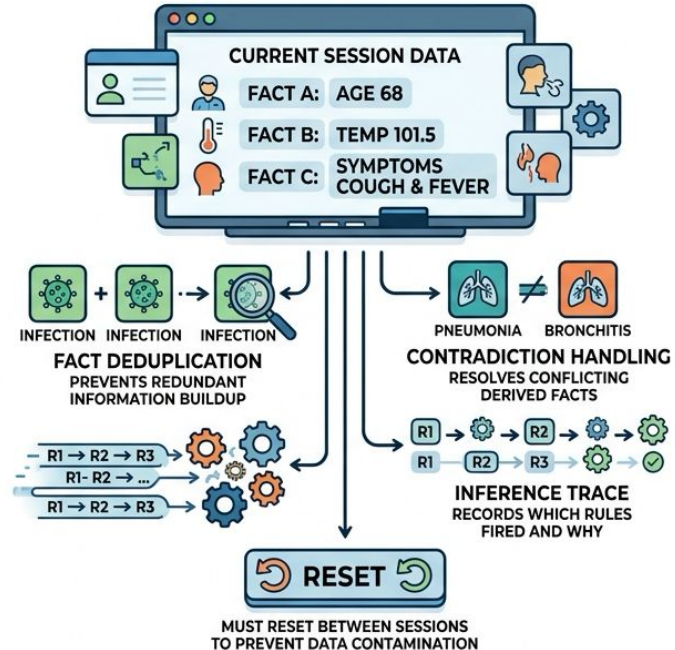
- Priority/salience ranks rules by assigned importance
- Specificity prefers rules with more matching conditions
- Recency favors recently added or modified rules
- Order of entry uses position in the knowledge base
- Production systems often layer multiple strategies

Heuristics & Working Memory

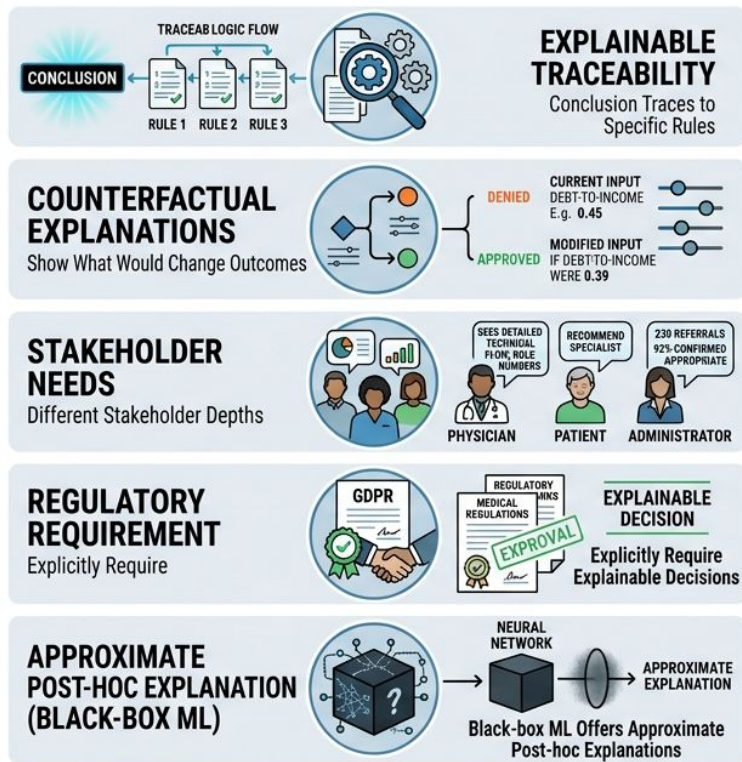
GOOD ENOUGH, FAST ENOUGH



THE SYSTEM'S SCRATCH PAD



Explanation and Traceability -- The Killer Advantage



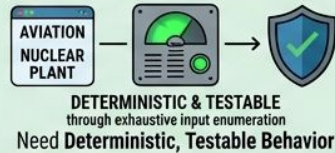
- Every conclusion traces to specific rules that produced it
- Counterfactual explanations show what would change outcomes
- Different stakeholders need different explanation depths
- Regulatory frameworks explicitly require explainable decisions
- Black-box ML offers only approximate post-hoc explanations

When Rules Win -- and When They Lose

REGULATORY DOMAINS



SAFETY-CRITICAL SYSTEMS



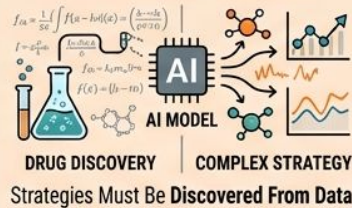
STABLE EXPERT KNOWLEDGE



LOW-DATA ENVIRONMENTS



UNKNOWN STRATEGIES



RAPIDLY EVOLVING DOMAINS

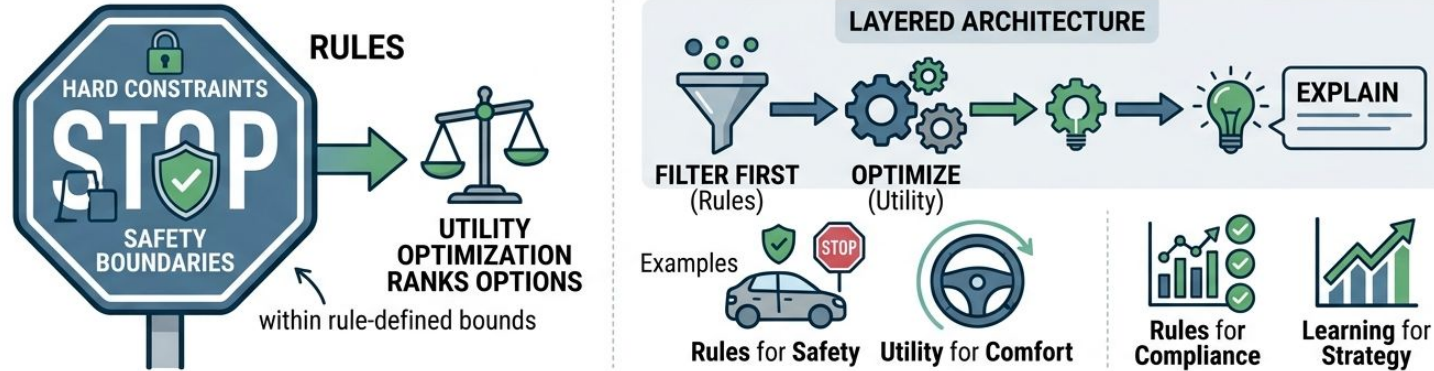


TOO COMPLEX FOR ENUMERATION



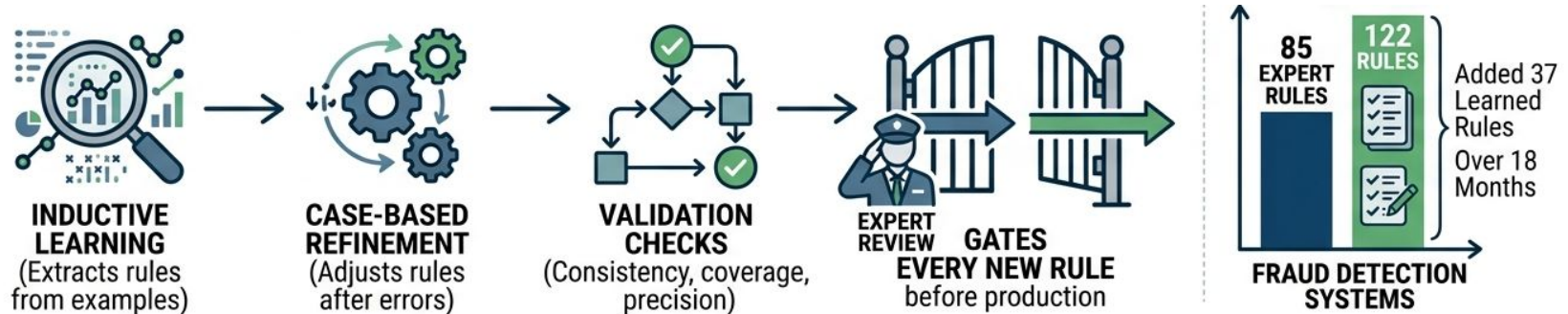
- Regulatory domains demand rule-level auditability
- Safety-critical systems need deterministic, testable behavior
- Stable expert knowledge translates directly into rules
- Low-data environments make learning impossible
- Rapidly evolving domains favor learning-based adaptation

Hybrid Architectures -- Rules Plus Optimization



- Rules define hard constraints and safety boundaries
- Utility optimization ranks options within rule-defined bounds
- Layered architecture: filter first, then optimize, then explain
- Autonomous vehicles use rules for safety, utility for comfort
- Financial trading uses rules for compliance, learning for strategy

Rule Learning -- Teaching Old Systems New Tricks



Key Takeaways

- Rule-based systems trade adaptability for transparency
 - Four components: knowledge base, inference engine, working memory, interpreter
 - Forward chaining explores; backward chaining verifies
 - Conflict resolution strategy shapes system behavior
 - Next chapter: what happens when agents learn their own rules
- 